

IU05-Assessment Assignment

Module Name	Database Design and Implementation
Course Name	Foundation Diploma in Software Engineering
Assignment Title	MetroCity College Record Management Review

Learner Name		Assessor Name
Eaint Nadi Maung		Arvinder Kaur
Date Issued	Completion Date	Submitted On
7/7/26	7/8/26	7/8/26

Plagiarism

Plagiarism covers both copying or paraphrasing with the source without proper understanding and citation. The source includes but is not limited to:

- Books, Journal Articles, Online resources
- Others Assignment or project
- Software program
- Images

The learners must submit their work without plagiarism while producing the evidence against the assessment criteria. They must apply proper citation and Harvard referencing. Their work must go through the Turnitin software to check for the similarity index. Please consult the relevant module instructor or mentor if you need any further advice.

Learner declaration	
I certify that the work submitted for this assignment is my own and research sources are fully acknowledged.	
Student signature: star	Date: 7/8/26

Contents

1. Project Background.....	3
2. Project Objective	3
3. Task 1 - Create a Standard Data Dictionary Using Excel	3
4. Task 2 - Generate Database Documentation Using dbForge	6
5. Task 3 - Manage Database Project Files Using Git and GitHub	10
6. Task 4 - Create a README.md File.....	14
7. Task 5 - Concept Questions	16
8. Task 6 - Prepare a Report with Screenshots of Each Task and Process	Error! Bookmark not defined.

- **Project Background**

MetroCity College manages student information across multiple departments, including student profiles, courses, enrollments, and examination results. The database system has already been designed, implemented, and tested in previous coursework. This project focuses on the final stage of the database lifecycle by documenting the completed database, organizing project files, and preparing the system for collaboration and future maintenance using professional database documentation and version control tools.

- **Project Objective**

The objective of this project is to professionally document and manage the completed MetroCity College Student Management Database. Learners will generate database documentation, create a data dictionary, use Git and GitHub for version control, prepare a professional README.md file, upload all project files to a GitHub repository, and produce a report demonstrating the complete documentation and collaboration process.

- **Task 1 - Create a Standard Data Dictionary Using Excel**

- **Step 1: Add Proper Data Dictionary Table Headers**

- Column Name
 - Description
 - Data Type
 - Size / Length
 - Null / Not Null

departments Table				
Column Name	Description	Data Type	Size/Length	Null / Not Null

- **Step 2: Fill in Details**

For each column in a table:

- Column Name: Exact column name from the database.
- Description: What this column stores in plain English.
- Data Type: SQL type (e.g., INT, VARCHAR, DATE).
- Size / Length: If applicable (e.g., VARCHAR (50), DECIMAL (10,2)).
- Null / Not Null: Write Null or Not Null.

- **Step 3: Repeat for all Tables**

Repeat the same process and complete the rest tables of Oakridge Database.

- Departments table
- Students table
- Courses table
- Enrolments table
- Exam Results table

- **Expected Output**

Table: departments

This table stores the academic departments available at MetroCity College and is used to organize students and courses.

Departments Table				
Column Name	Description	Data Type	Size/Length	Null / Not Null
department_id	Unique identifier for each department.	INT	11	NOT NULL
department_name	Name of the academic department.	VARCHAR	100	NOT NULL

Table: students

This table stores each student's personal, contact, and department information.

Students Table				
Column Name	Description	Data Type	Size/Length	Null / Not Null
student_id	Unique identifier for each student.	INT	11	NOT NULL
first_name	Student's first name.	VARCHAR	50	NOT NULL
last_name	Student's last name.	VARCHAR	50	NOT NULL
gender	Student's gender.	ENUM('Male','Female')	----	NOT NULL
date_of_birth	Student's date of birth.	DATE	----	NOT NULL
email	Student's email address.	VARCHAR	100	NOT NULL
phone	Student's contact phone number.	VARCHAR	15	NOT NULL
department_id	Reference to the department the student belongs to.	INT	11	NOT NULL

Table: courses

This table contains the courses offered by the college, including course details and the department that provides them.

Courses Table				
Column Name	Description	Data Type	Size/Length	Null / Not Null
course_id	Unique identifier for each course.	INT	11	NOT NULL
course_code	Unique code assigned to the attend course (e.g., IT101)	VARCHAR	20	NOT NULL
course_name	Name of the course.	INT	100	NOT NULL
credit_hours	Number of credit hours assigned to the course.	INT	11	NOT NULL
credit_hours	Reference to the department offering the course.	DATE	11	NOT NULL

Table: enrollments

This table records the courses that students register for each semester and academic year.

Enrollments Table				
Column Name	Description	Data Type	Size/Length	Null / Not Null
enrollment_id	Unique identifier for each enrollment record.	INT	11	NOT NULL
student_id	Reference to the enrolled student.	INT	11	NOT NULL
course_id	Reference to the enrolled course.	INT	11	NOT NULL
semester	Academic semester in which the student is enrolled.	VARCHAR	20	NOT NULL
academic_year	Academic year of the enrollment.	INT	11	NOT NULL
enrollment_date	Date the student enrolled in the course.	DATE	----	NOT NULL

Table: exam_resulta

This table stores students' exam performance, including the exam date, score, and final grade for each enrollment.

Exam_Results Table				
Column Name	Description	Data Type	Size/Length	Null / Not Null
result_id	Unique identifier for each exam result.	INT	11	NOT NULL
enrollment_id	Reference to the student's enrollment record.	INT	11	NOT NULL
exam_date	Date the examination was conducted.	DATE	----	NOT NULL
score	Score obtained by the student in the examination.	DECIMAL	5,2	NOT NULL
grade	Grade awarded based on the student's score.	CHAR	2	NOT NULL

- **Step 4: Review and Finalize**
 - Check that column names match the database.
 - Make sure descriptions are clear and understandable.
 - Verify that data types, sizes, null rules, and constraints are correct.
 - Save the Excel file: File → Save.

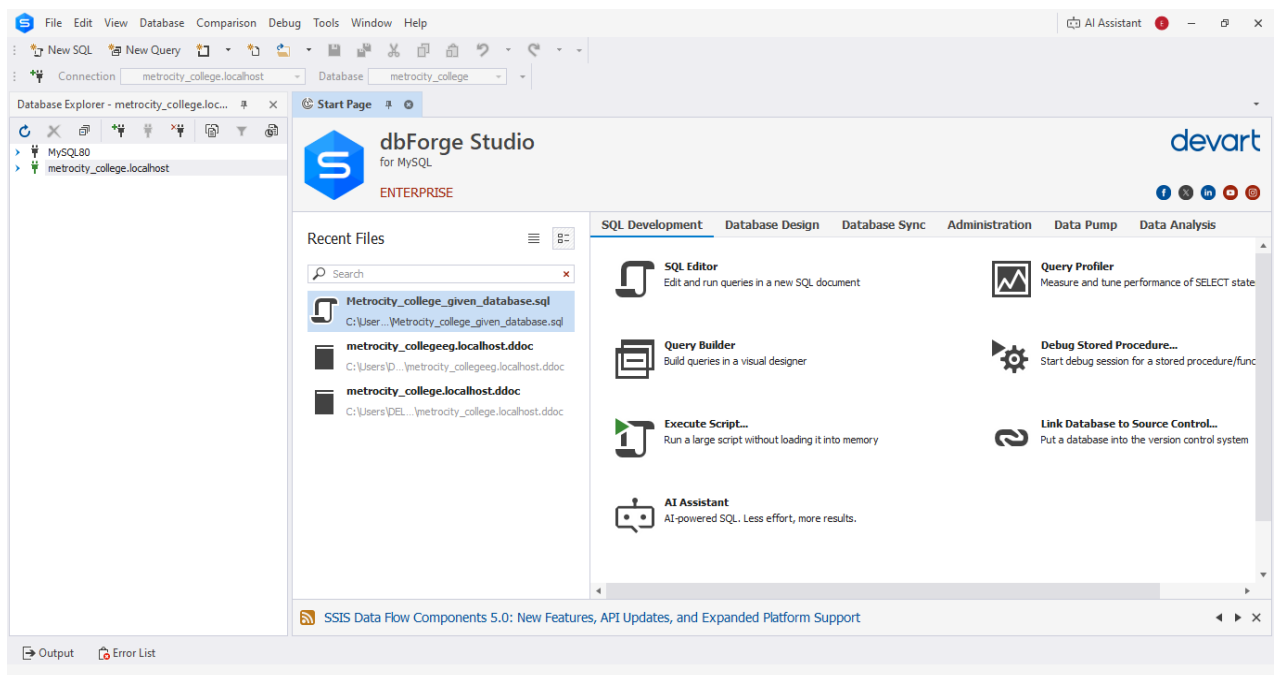
Store it in the project's documentation folder.

Expected Output

Name	Status	Date modified	Type	Size
 A05_MetroCity_Updated_Template		7/8/2026 3:44 PM	Microsoft Word D...	492 KB
 Metrocity_college_given_database		7/6/2026 7:20 AM	SQL Text File	5 KB
 MetroCity_Data_Dictionary		7/8/2026 11:48 AM	Microsoft Excel W...	15 KB

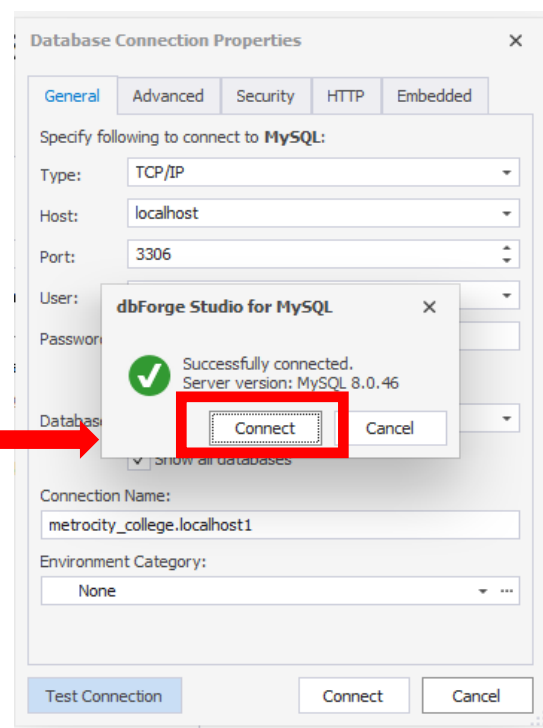
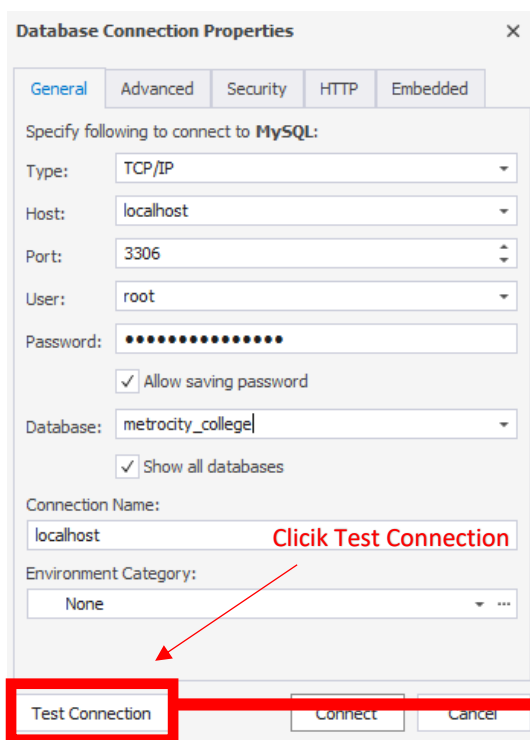
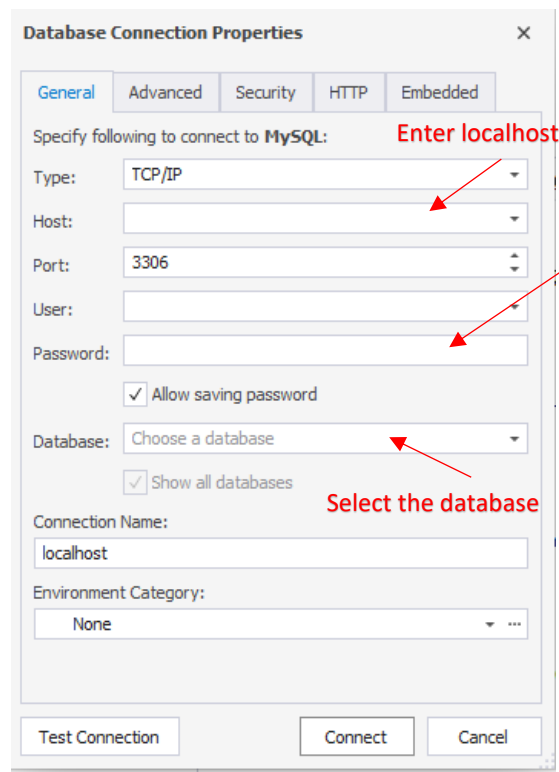
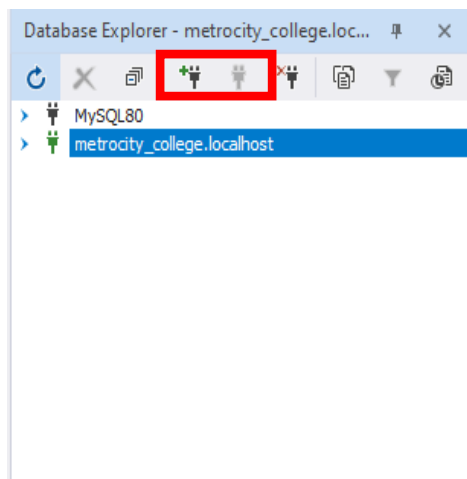
- **Task 2 - Generate Database Documentation Using dbForge**

- **Step 1: Launch and Open dbForge Studio**



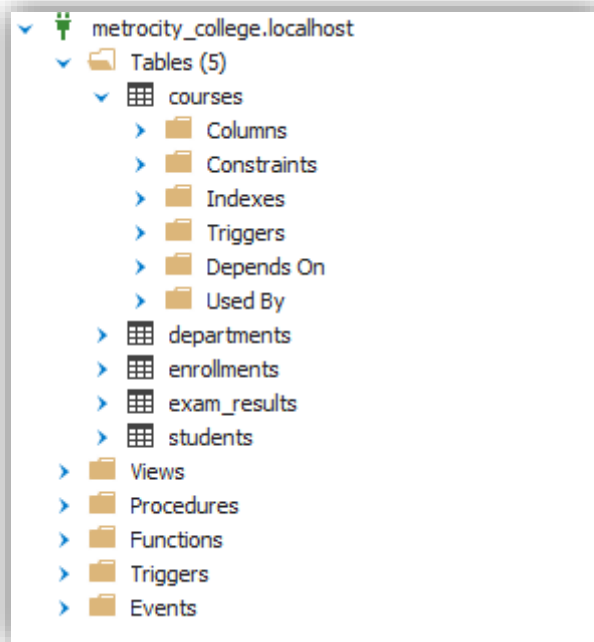
- **Step 2: Create a New Database Connection**

- Click Database → New Connection
- A Database Connection Properties Dialog Will Appear

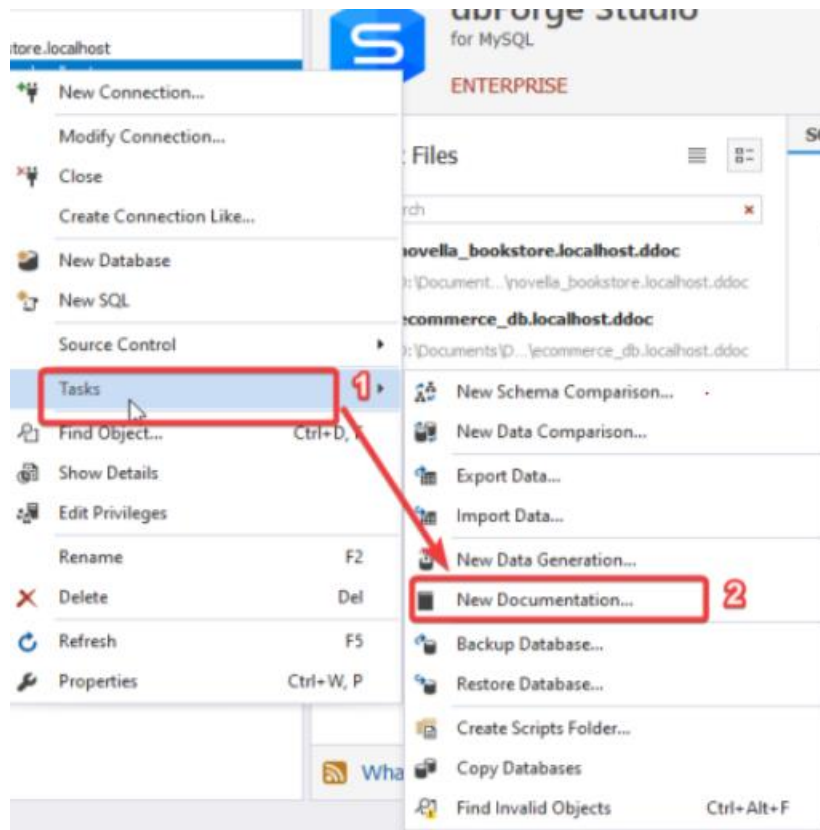


After connecting:

- Locate your database in the Database Explorer
- Expand the database to see:
 - Tables
 - Views
 - Procedures

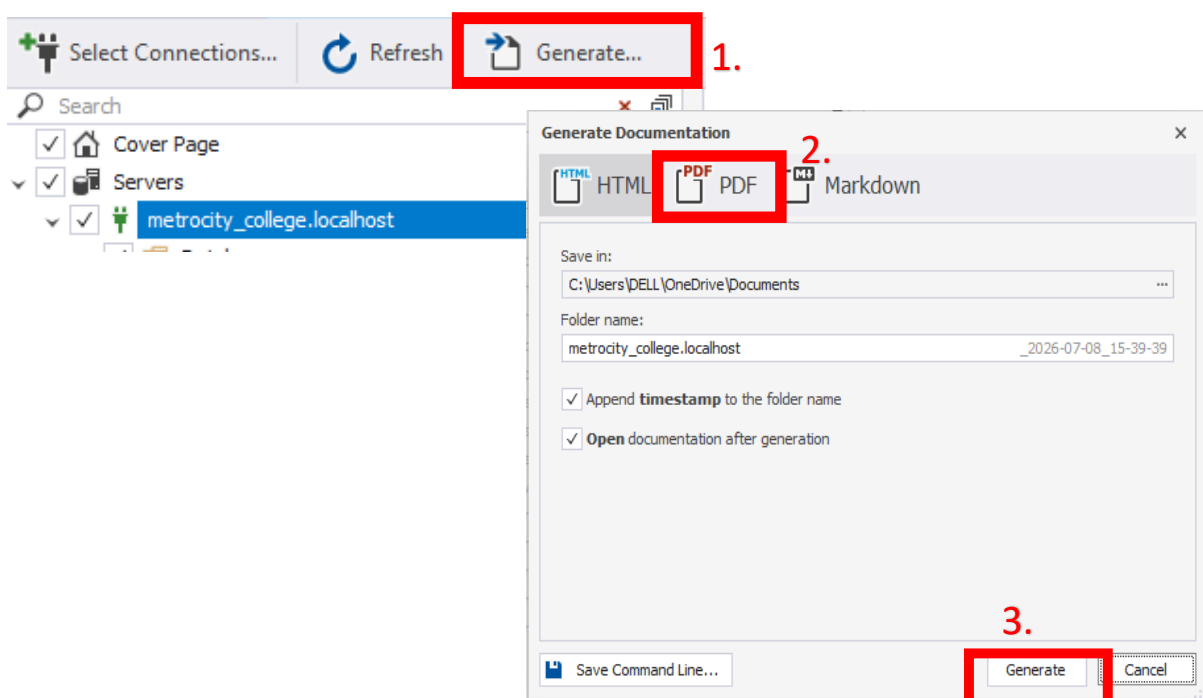


- **Step 3: Open the Database Documentation Tool**
 - In the Database Explorer, right-click on your database.
 - Select Tasks → New Documentation



- **Step 4: Generate Documentation**

- Locate the Generate button in your documentation tool.
- Click it to create the database documentation.
- Choose PDF format



Expected Output

Name	Status	Date modified	Type	Size
A05_MetroCity_Updated_Template		7/8/2026 3:44 PM	Microsoft Word D...	492 KB
Metrocity_college_given_database		7/6/2026 7:20 AM	SQL Text File	5 KB
MetroCity_Data_Dictionary		7/8/2026 11:48 AM	Microsoft Excel W...	15 KB
MetroCity_DB_Documentation		7/8/2026 11:43 AM	Chrome PDF Doc...	3,621 KB

• Task 3 - Manage Database Project Files Using Git and GitHub

• Step 1: Create a Project Folder

- Create a new folder anywhere on your computer.
- This folder will store all the files related to your database.

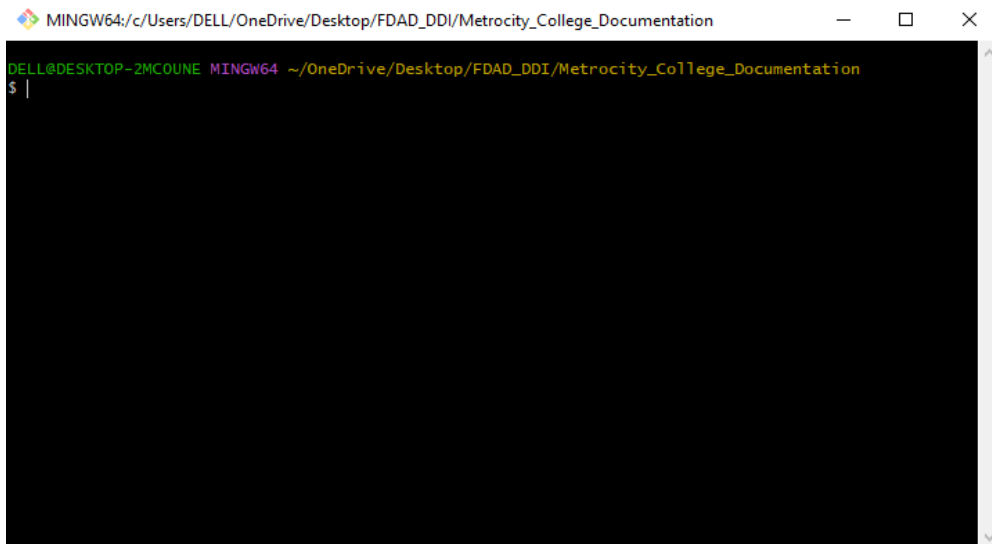
Name	Status	Date modified	Type	Size
IU03_ASSIGNMENT		7/4/2026 3:09 AM	File folder	
IU4_ASSIGNMENT		7/6/2026 9:33 PM	File folder	
Metrocity_College_Documentation		7/8/2026 4:32 PM	File folder	

• Step 2: Store the Project Files inside the Folder

Name	Status	Date modified	Type	Size
A05_MetroCity_Updated_Template		7/8/2026 4:35 PM	Microsoft Word D...	524 KB
ER-diagram and Schema		6/28/2026 11:50 AM	Chrome PDF Doc...	575 KB
Metrocity_college_given_database		7/6/2026 7:20 AM	SQL Text File	5 KB
MetroCity_Data_Dictionary		7/8/2026 11:48 AM	Microsoft Excel W...	15 KB
MetroCity_DB_Documentation		7/8/2026 11:43 AM	Chrome PDF Doc...	3,621 KB

• Step 3: Open your Project Folder with Git

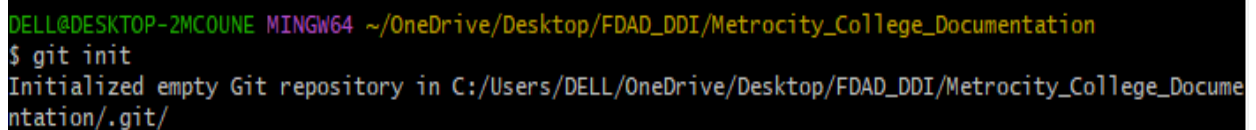
Open Git Bash: Right-click inside your Metrocity_College_Documentation folder and select "Open Git Bash Here."



```
MINGW64: c:/Users/DELL/OneDrive/Desktop/FDAD_DDI/Metrocity_College_Documentation
DELL@DESKTOP-2MCOUNE MINGW64 ~/OneDrive/Desktop/FDAD_DDI/Metrocity_College_Documentation
$ |
```

- **Step 4: Initialize Git (git init)**

Before starting version control, you need to initialize a Git repository in your project folder. Type this command: **git init**



```
DELL@DESKTOP-2MCOUNE MINGW64 ~/OneDrive/Desktop/FDAD_DDI/Metrocity_College_Documentation
$ git init
Initialized empty Git repository in C:/Users/DELL/OneDrive/Desktop/FDAD_DDI/Metrocity_College_Documentation/.git/
```

- **Step 5: Check the status of your files (git status)**

After initializing the repository, check the current status of your files using: **git status**

This command shows:

- Which files are untracked (new files that Git is not yet tracking)
- Which files have been modified since the last commit
- Any files that are already staged and ready to commit

```

DELL@DESKTOP-2MCOUNE MINGW64 ~/OneDrive/Desktop/FDAD_DDI/Metrocity_College_Documentation (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    A05_MetroCity_Updated_Template.docx
    ER-diagram and Schema.pdf
    MetroCity_DB_Documentation.pdf
    MetroCity_Data_Dictionary.xlsx
    Metrocity_college_given_database.sql

nothing added to commit but untracked files present (use "git add" to track)

```

- **Step 6: Add Files to Staging Area (git add .)**

To tell Git which files you want to include in your next commit, you need to add them to the staging area. You can do this using: **git add .**

- The . (dot) adds all files and subfolders in the current directory to the staging area.
- Alternatively, you can add files individually by specifying their names, for example:

git add Metrocity_college_Dictionary.xlsx , git add ERD/metrocity_ERD.png

```

DELL@DESKTOP-2MCOUNE MINGW64 ~/OneDrive/Desktop/FDAD_DDI/Metrocity_College_Documentation (master)
$ git add .
warning: in the working copy of 'Metrocity_college_given_database.sql', LF will be replaced by CRLF
the next time Git touches it

```

- **Step 7: Commit Your Changes (git commit -m "Commit Message")**

Once your files are added to the staging area, the next step is to commit the changes. A commit saves a snapshot of your current files in the Git repository so you can track changes over time.

Use this command: **git commit -m "Commit Message"**

```

DELL@DESKTOP-2MCOUNE MINGW64 ~/OneDrive/Desktop/FDAD_DDI/Metrocity_College_Documentation (master)
$ git commit -m "Add metrocity college database project"
[master (root-commit) 135d183] Add metrocity college database project
5 files changed, 159 insertions(+)
create mode 100644 A05_MetroCity_Updated_Template.docx
create mode 100644 ER-diagram and Schema.pdf
create mode 100644 MetroCity_DB_Documentation.pdf
create mode 100644 MetroCity_Data_Dictionary.xlsx
create mode 100644 Metrocity_college_given_database.sql

```

- **Step 8: Connect to a remote repository (GitHub, GitLab, etc.)**

Top repositories

New

Find a repository...

eaintstar123-dot/metrocity_college

Click the NEW button

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner *

eaintstar123-dot

Repository name *

Metrocity_college_IU5_Project

Metrocity_college_IU5_Project is available.

Great repository names are short and memorable. How about [bookish-octo-system](#)?

Description

0 / 350 characters

Add a clear repository name

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off ☐

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

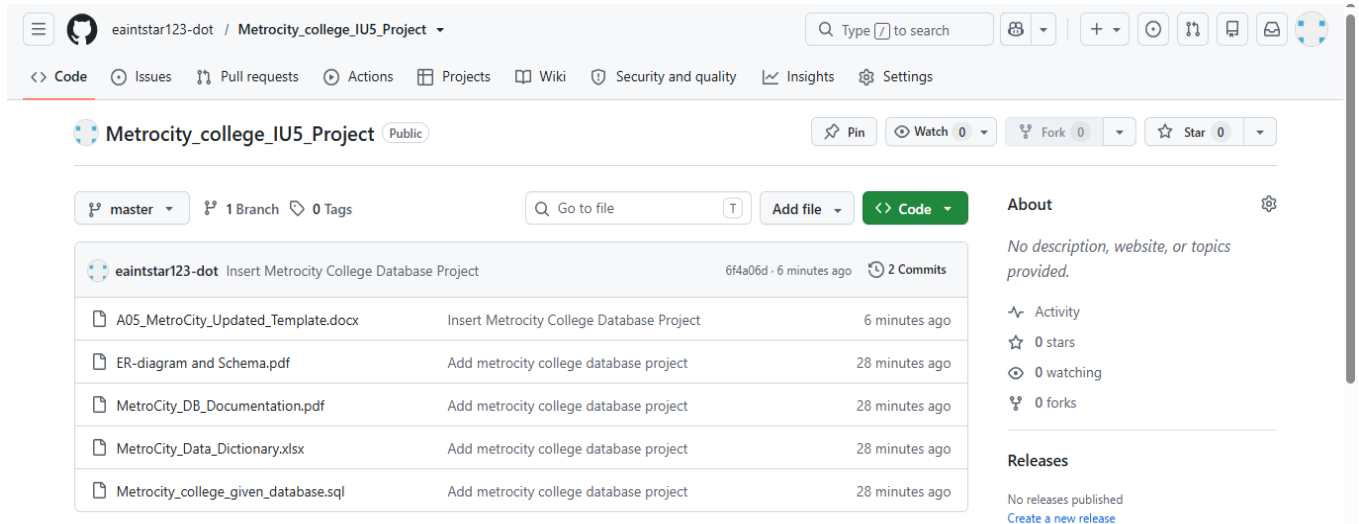
Licenses explain how others can use your code. [About licenses](#)

No license

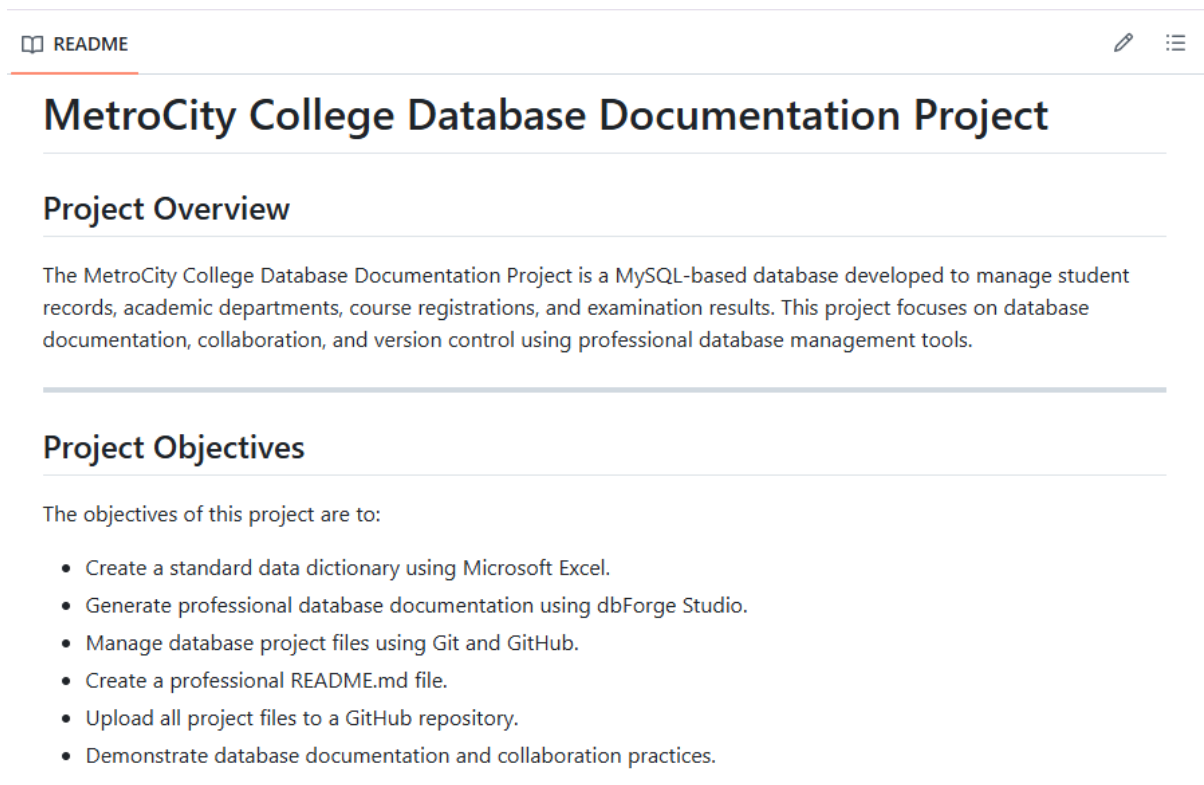
Create repository

The GitHub repository was successfully created and configured with the project name, visibility settings, and initial README file. The repository was then prepared for uploading the MetroCity College database project files.

Expected Output



• Task 4 - Create a README.md File



Database Description

The database manages academic information for MetroCity College. It stores student details, department information, course records, student enrollments, and examination results. The database uses primary keys and foreign keys to maintain data integrity and relationships between tables.

Database Tables

- Departments
- Students
- Courses
- Enrollments
- Exam_Results

🔗 Main Features

- Manage department information.
- Store student records.
- Manage course details.
- Record student enrollments.
- Store examination results, scores, and grades.
- Maintain relationships using primary and foreign keys.
- Support database documentation and collaboration.

Tools and Software Used

- MySQL
- MySQL Workbench
- dbForge Studio for MySQL
- Microsoft Excel
- Git
- GitHub
- Microsoft Word

🔗 GitHub Repository Information

Repository Name

Metrocity_College_IU5_Project

Repository Purpose

This repository stores all project files, tracks project changes using Git, and provides version control through GitHub.

Project Folder Structure

```
Metrocity_College_IU5_Project/  
├── README.md  
├── Metrocity_college_given_database.sql  
├── MetroCity_Data_Dictionary.xlsx  
├── MetroCity_DB_Documentation.pdf  
├── ER-diagram and Schema.pdf  
└── A05_MetroCity_Updated_Template.docx
```



Author

Eaint Nadi Maung

• Task 5 - Concept Questions

1. Explain how database documentation contributes to database scalability, maintainability, and future system enhancement.

Database documentation provides a clear record of the database structure, including tables, relationships, keys, and constraints. It makes the database easier to understand, maintain, and update. Good documentation also supports scalability by allowing developers to add new features or expand the database without affecting the existing structure.

2. Discuss the importance of metadata in managing complex database systems and ensuring data consistency.

Metadata describes the structure of a database, such as table names, column names, data types, and relationships. It helps developers understand how data is organized and ensures that data is stored consistently. Proper metadata also reduces errors and improves database management.

3. Compare manual database documentation and automated documentation generated by tools such as dbForge.

Manual documentation is created by the developer and allows detailed explanations and custom notes. However, it requires more time and can become outdated. Automated documentation generated by dbForge is faster, more accurate, and automatically includes tables, relationships, keys, and constraints, making it easier to maintain.

4. Explain how Git and GitHub support collaborative database development and reduce risks during database updates.

Git tracks changes made to database project files, while GitHub stores those changes in an online repository. They allow multiple developers to collaborate, maintain version history, and restore previous versions if mistakes occur. This reduces the risk of data loss and helps manage database updates efficiently.

5. Analyze the potential problems that may occur when database projects are managed without proper version control and documentation practices.

Without version control and documentation, developers may overwrite each other's work, lose important changes, or make inconsistent updates. Poor documentation can make the database difficult to understand and maintain, leading to errors, delays, and challenges when adding new features or troubleshooting problems.

GitHub Repository

https://github.com/eaintstar123-dot/Metrocity_College_IU5_Project